



۸۱۰۱۰۱۴۹۲



تمرین ۱

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی

سوال ۱

الف) مقایسه بین ps و cs

روش ps برای داده‌های رگباری خوب است. (ترافیک های رگباری ($bursty$)، یعنی کاربران اغلب اوقات خاموش هستند و فقط بعضی اوقات، داده می‌فرستند.) چون می‌تواند از مزیت اشتراک منابع استفاده کند. همچنین در این روش $call\ setup$ اولیه را نداریم و الگوریتم ها ساده تر هستند. از طرفی اگر شبکه دچار ازدحام $congestion$ شوند، تاخیر شبکه زیاد می‌شود.

برای مالتی مدیا و استریمینگ و تماس تلفنی، باید تاخیر کرانمند (و یک حداقل $bandwidth$) داشته باشیم که برای آنها cs روش بهتری است. (چون لینک به صورت اختصاصی است و نه اشتراکی و اون حداقل ها تضمین می‌شود. و یک نرخ $stable$ برای انتقال داده به ما می‌دهد [۱])

در واقع در cs تاخیر ها کمتر است و اتفاقاتی مثل منتظر ماندن داده‌ها در صف، یا مشکل $congestion$ و ازدحام داده و در نتیجه $packet\ drop$ مواجه نمی‌شویم [۱]. (چون مسیر اختصاصی داریم و حتی اگر برای مدتی کاری نداشته باشیم، به فرد دیگری داده نمی‌شود)

پس اگر دنبال کارایی قابل پیش‌بینی و مطمئن و درستی ترتیب ارسال داده‌ها و راحتی فورواردیگ (به دلیل $exact\ match$ و نبود سربار حافظه آدرس مقصد در هدر) هستیم، و یا می‌خواهیم داده‌ها یا ترافیک زیاد را هندل کنیم، باید به سراغ cs برویم. درواقع هزینه $call\ setup$ روی حجم و تعداد بسته های ارسالی، سرشکن می‌شود و اینگونه به صورت میاتگین برای هر کدام میزان کمی تاخیر در نظر گرفته می‌شود. (در صورتی که اگر تعداد بسته ها کم بود، برای هر بسته، تاخیر $call\ setup$ زیادی میفتاد)

در مقابل اگر داده‌ها کم است و می‌خواهیم از لینک‌ها با کارایی $utilization$ بالا استفاده کنیم و هزینه سربار ذخیره $state$ در هر روتر یا $call\ setup$ اولیه را ندهیم، بهتر است از ps استفاده کنیم.

ب) مقایسه بین FDM و TDM

یک سناریو خاص که TDM در آن بهتر است، این است که دیتای ما کوچک باشد و بتواند در یک اسلات ارسال شود. داده‌ای که در یک اسلات و در مدت کوتاه در TDM ارسال می‌شود، به دلیل تقسیم شدن و کاهش $bandwidth$ در FDM ، باید مدت زمان بیشتری را سپری کند.

پس به نوعی می‌توان گفت که TDM برای داده‌های $bursty$ که هر چند وقت یکبار رخ می‌دهند و مدت زمان on کوتاه است و داده‌های کوچکی هستند، بهتر است. در مقابل، برای کاربردهای $real-time$ و $streaming$ و $gaming$ که به طور مستمر باید داده‌ها آپدیت شوند، چون در FDM یک حداقل $bandwidth$ داریم، این راه برای آن کاربردها بهتر است.

اینطوری که سرچ کردم [۲]، برای جداسازی فرکانس‌ها در روش FDM ، باید تجهیزات خاصی استفاده کنیم که این هم هزینه بر دار است هم باعث اتلاف بخشی از پهنای باند نیز می‌شود. از طرفی هنوز هم مشکل نویز و تاثیر کانال‌های مجاور روی هم در FDM هست. در صورتی که در TDM هم مشکل تقسیم پهنای باند و گرانی تجهیزات آن و اتلاف بخشی از باند را نداریم و می‌توانیم از کل ظرفیت باند استفاده کنیم، هم مشکل نویز نداریم چون در هر لحظه فقط یک نوع داده در حال انتقال است و در کل با تجهیزات ارزان‌تر و راحت‌تر این کار صورت می‌گیرد.

سوال ۳

تاخیر کل که مجموع تاخیر ارسال و تاخیر انتشار محاسبه می‌کنیم.^۱ برای تاخیر انتقال (ارسال) داده، طول بسته L و همچنین نرخ انتقال $R = \text{transmission rate}$ روی آن تاثیر دارند. پس داریم:

$$\text{transmission delay} = \frac{L(b)}{R(bps)} = \frac{1000 \times 8}{2 \times 10^6} = 4 \times 10^{-3} \text{sec} = 4ms$$

برای تاخیر انتشار روی لینک، طول بسته و نرخ ارسال (انتقال) داده، روی آن تاثیر ندارند و فقط فاصله و سرعت انتشار موج در آن مدیا، روی سرعت انتشار تاثیر دارند. پس به طور کلی:

$$\text{propagation delay} = \frac{d(m)}{s(m/s)}$$

در مثال ما، این مقدار برابرست با:

^۱ برداشت خودم از سوال، این بود که فقط تاخیر انتشار مد نظر است. برای همین، طول بسته و نرخ انتقال transmission rate در آن تاثیر ندارد، اما از TA تمرین، آقای کوروش سجادی، پرسیدم و ایشان گفتند که منظور کل تاخیر انتقال + انتشار داده است.

$$propagation\ delay = \frac{2500 \times 1000}{2.5 \times 10^8} = 10^{-2} sec = 10ms$$

پس تاخیر کل انتقال و انتشار داده برابرست با:

$$total\ delay = transmission\ delay + propagation\ delay = 4ms + 10ms = 14ms$$

پس تاخیر انتشار به طول بسته یا نرخ انتقال بستگی ندارد اما تاخیر انتقال به طول بسته و نرخ انتقال بستگی دارد. پس این دو در تاخیر کل، تاثیر گذار هستند.

سوال ۵

(الف)

فاصله بین *San Francisco* تا *New York* به طور خط مستقیم برابر 4,129 کیلومتر است. [۳] از طرفی، سرعت انتشار موج در مدیا، از آنجایی که ذکر نشده، به صورت *default* و برابر $2 \times 10^8 (m/s)$ در نظر می‌گیریم. پس برای محاسبه d_{prop} داریم:

$$propagation\ delay = \frac{distance}{propagation\ speed} = \frac{4.129(km) \times 1000}{2 \times 10^8 (m/s)} \approx 2.06 \times 10^{-2} sec$$

همانطور که میدانیم *bandwidth* نرخ انتقال تعداد بیت بر ثانیه به روی لینک است. از طرفی کل مدت زمان انتشار هم از طریق d_{prop} بدست آمده که درواقع مدت زمانیست که یک بیت در حال انتشار روی لینک است. با ضرب این دو مولفه در هم، حداکثر حجم بیت داده ممکن که می‌تواند روی لینک قرار داشته باشد، بدست می‌آید و خواهیم داشت:

$$bandwidth \times d_{prop} = 500 \times 10^6 \times 2.06 \times 10^{-2} = 10.3 \times 10^6 (b) = 10.3(Mb)$$

به تعبیری، این حداکثر ظرفیت لینک به بیت است.

(ب)

چون $0.8Mb$ در حال انتشار است و حداکثر ظرفیت لینک هم $10.3Mb$ است، پس کل آن دیتای $0.8Mb$ همزمان می‌تواند روی لینک قرار بگیرد.

(ج)

باید مدت زمانی که یک بیت به روی لینک انتقال میابد را در سرعت انتشار آن در مدیا ضرب کنیم تا عرض بیت در مدیا را بدست آوریم.

برای مدت زمان انتقال یک بیت روی لینک، داریم:

$$Time\ per\ bit = \frac{1}{bandwidth} = \frac{1}{500 \times 10^6 (bits/sec)} = 2 \times 10^{-9} sec/bit$$

حال اگر این ثانیه به بیت را ضرب در سرعت انتشار بر حسب متر بر ثانیه کنیم، حاصل بر حسب متر بر بیت $meters\ per\ bit$ خواهد بود. پس داریم:

$$2 \times 10^{-9} (s/b) \times 2 \times 10^8 (m/s) = 4 \times 10^{-1} (m/b) = 0.4\ meters\ per\ bit$$

پس عرض هر بیت به بیانی ۴۰ سانتی‌متر است. چون ۴۰ سانتی‌متر به هر بیت اختصاص یافته است.

سوال ۷

(الف)

$$d_{prop} = \frac{m(meters)}{s(meters/sec)} = \frac{m}{s}(sec)$$

(ب)

$$d_{trans} = \frac{L(bit)}{R(bit/sec)} = \frac{L}{R}(sec)$$

(ج)

$$end - to - end\ delay = d_{prop} + d_{trans} = \frac{m}{s} + \frac{L}{R}(sec)$$

(د)

در زمان $t = d_{trans}$ آخرین بیت منطقی تازه روی لینک قرار گرفته و ابتدای لینک است.

(ه)

فرض کنیم $d_{drop} > d_{trans}$ است. از آنجایی که d_{drop} در واقع تاخیر طی کردن یک بیت از مبدا به مقصد روی لینک است، پس در زمان $t = d_{trans}$ ، بیت اول باید در میانه مسیر باشد.

(و)

فرض کنیم $d_{drop} < d_{trans}$ است. پس اگر به صورت تقریبی فرض کنیم که در زمان $t \approx 0$ بیت اول روی لینک قرار گرفته، در زمان $t = d_{trans}$ چون مدت زمان d_{prop} را پیموده، حتما به مقصد رسیده است.^۲

(ز)

$$d_{prop} = d_{trans} \implies \frac{m}{s} = \frac{L}{R} \implies \frac{m}{2.5 \times 10^8 (\text{meter/sec})} = \frac{1500 \times 8 (\text{bit})}{10 \times 10^6 (\text{bit/sec})}$$

$$\implies m = 3 \times 10^5 (\text{meters}) = 300 (\text{km})$$

سوال ۸

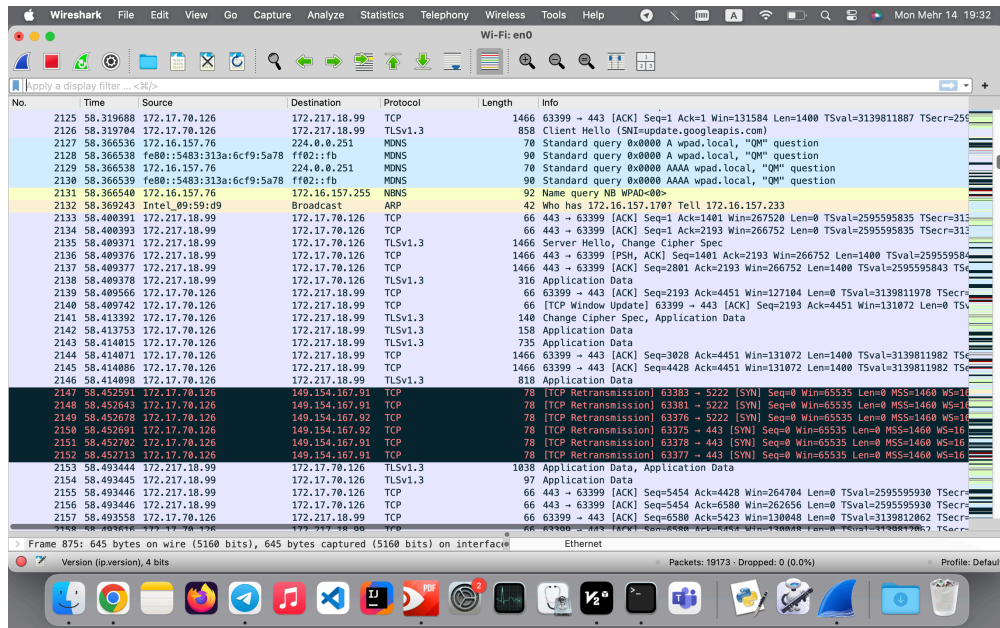
(۱)

همانطور که در شکل های ۱ و ۲ مشاهده می‌کنید، این پروتکل‌ها مورد استفاده بوده‌است. پروتکل‌هایی نظیر: $TCP, UDP, TLSv1.3, HTTP, NBNS, MDNS, ARP, BROWSER, QUIC, DNS, IGMPv2$ و غیره.

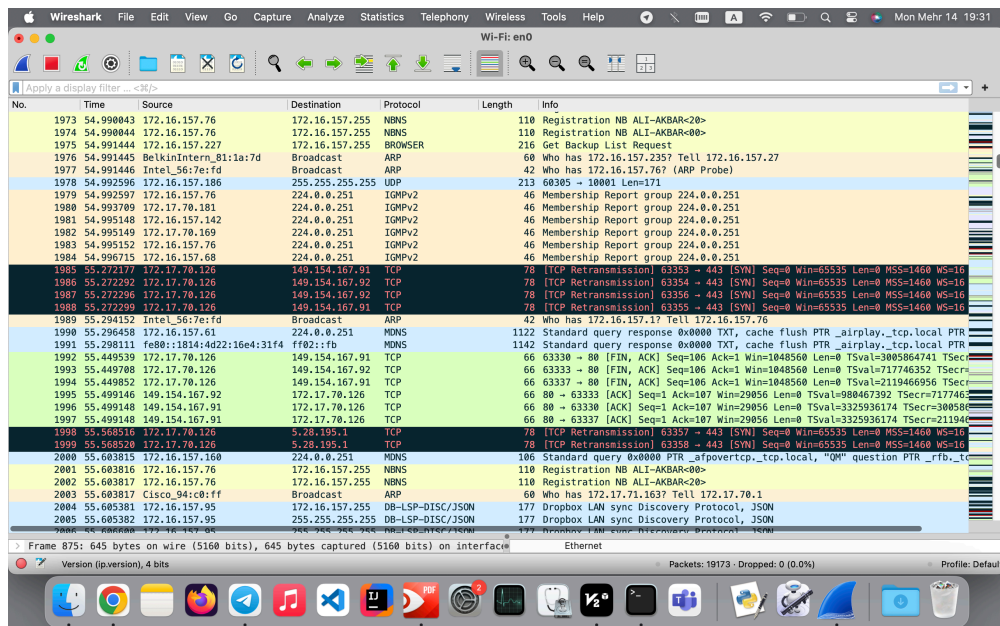
(۲)

همانطور که می‌بینید، تایم $request$ و $response$ در شکل ۳ مشخص شده‌است که تقریباً برابر با 0.35 sec است.

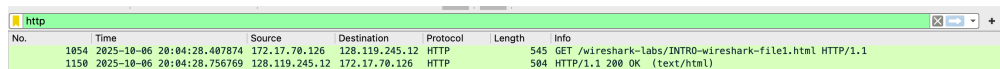
^۲ اگر به طور دقیق تر بخواهیم بگوییم، بیت اول پس از $\frac{1}{R}$ ثانیه، وارد لینک می‌شود. پس در صورتی که $d_{drop} + \frac{1}{R} \leq d_{trans}$ باشد، بیت اول به مقصد رسیده است. یعنی اگر خیلی دقیق نگاه کنیم و داشته باشیم $d_{drop} + \frac{1}{R} > d_{trans}$ آنگاه بیت اول هنوز در میانه لینک (نزدیک به انتها) است.



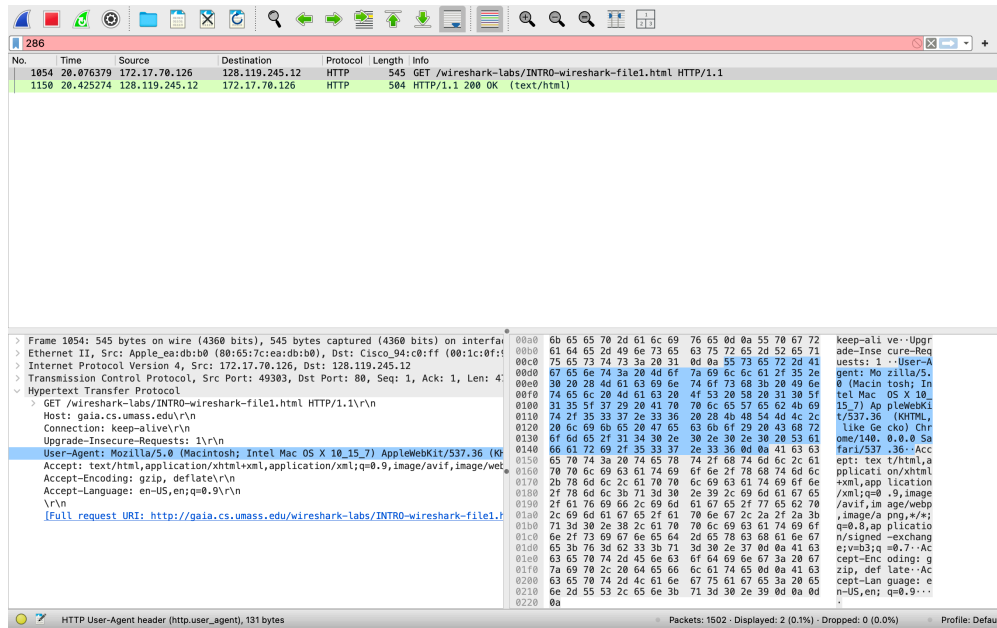
شکل ۱: Protocols1



شکل ۲: Protocols2



شکل ۳: Time - difference



شکل ۴: User-agent

(۳)

همانطور که در شکل ۳ می‌توانید مشاهده کنید، آدرس فرستنده (ما) برابر 172.17.70.126 است و آدرس IP مقصد (سرور وب) برابر 128.119.245.12 است.

(۴)

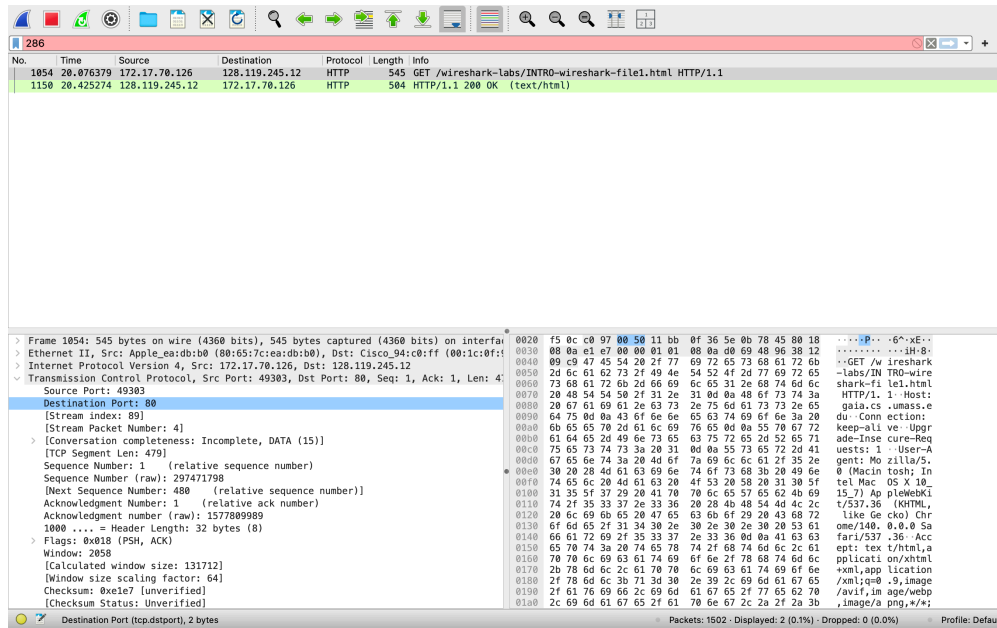
همانطور که در شکل ۴ مشاهده می‌کنید، از بروزر موزیلا ۵ استفاده شده است.

(۵)

همانطور که در شکل ۵ مشاهده می‌کنید، port مقصد در پروتکل HTTP، ۸۰ است، همانطور که توقع داشتیم.

(۶)

همانطور که در صفحات آخر در شکل ۶ مشاهده می‌کنید، این بخشی از اطلاعات HTTP request است. و همانطور که در شکل ۷ مشاهده می‌کنید، این بخشی از اطلاعات HTTP response است.



شکل ۵: Destination port

/var/folders/qy/v1g2pryd24x2x6ptc9xgsgsm0000gn/T/wireshark_Wi-FiNFU5D3.pcapng 1502 total packets, 1 shown

No.	Time	Source	Destination	Protocol	Length	Info
1054	20.076379	172.17.70.126	128.119.245.12	HTTP	545	GET /

wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1
 Frame 1054: 545 bytes on wire (4360 bits), 545 bytes captured (4360 bits) on interface en0, id 0
 Ethernet II, Src: Apple_ea:db:b0 (80:65:7c:ea:db:b0), Dst: Cisco_94:c0:ff (00:1c:0f:94:c0:ff)
 Internet Protocol Version 4, Src: 172.17.70.126, Dst: 128.119.245.12
 Transmission Control Protocol, Src Port: 49303, Dst Port: 80, Seq: 1, Ack: 1, Len: 479
 Source Port: 49303
 Destination Port: 80
 [Stream index: 89]
 [Stream Packet Number: 4]
 [Conversation completeness: Complete, WITH_DATA (31)]
 [TCP Segment Len: 479]
 Sequence Number: 1 (relative sequence number)
 Sequence Number (raw): 297471798
 [Next Sequence Number: 480 (relative sequence number)]
 Acknowledgment Number: 1 (relative ack number)
 Acknowledgment number (raw): 1577809989
 1000 = Header Length: 32 bytes (8)
 Flags: 0x018 (PSH, ACK)
 Window: 2058
 [Calculated window size: 131712]
 [Window size scaling factor: 64]
 Checksum: 0xe1e7 [unverified]
 [Checksum Status: Unverified]
 Urgent Pointer: 0
 Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
 [Timestamps]
 [SEQ/ACK analysis]
 TCP payload (479 bytes)
 Hypertext Transfer Protocol
 GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1\r\n
 Host: gaia.cs.umass.edu\r\n
 Connection: keep-alive\r\n
 Upgrade-Insecure-Requests: 1\r\n
 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36\r\n
 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7\r\n
 Accept-Encoding: gzip, deflate\r\n
 Accept-Language: en-US,en;q=0.9\r\n
 \r\n
 [Response in frame: 1150]
 [Full request URI: http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html]

شکل ۶: HTTP request

/var/folders/qy/v1g2pryd24x2x6ptc9xgsgsm0000gn/T/wireshark_Wi-FiNFU5D3.pcapng 1502 total packets, 1 shown

No.	Time	Source	Destination	Protocol	Length	Info
1150	20.425274	128.119.245.12	172.17.70.126	HTTP	504	HTTP/1.1 200 OK (text/html)

Frame 1150: 504 bytes on wire (4032 bits), 504 bytes captured (4032 bits) on interface en0, id 0

Ethernet II, Src: Cisco_94:c0:ff (00:1c:0f:94:c0:ff), Dst: Apple_ea:db:b0 (80:65:7c:ea:db:b0)

Internet Protocol Version 4, Src: 128.119.245.12, Dst: 172.17.70.126

Transmission Control Protocol, Src Port: 80, Dst Port: 49303, Seq: 1, Ack: 480, Len: 438

Source Port: 80

Destination Port: 49303

[Stream index: 89]

[Stream Packet Number: 6]

[Conversation completeness: Complete, WITH_DATA (31)]

[TCP Segment Len: 438]

Sequence Number: 1 (relative sequence number)

Sequence Number (raw): 1577809989

[Next Sequence Number: 439 (relative sequence number)]

Acknowledgment Number: 480 (relative ack number)

Acknowledgment number (raw): 297472277

1000 = Header Length: 32 bytes (8)

Flags: 0x018 (PSH, ACK)

Window: 235

[Calculated window size: 30080]

[Window size scaling factor: 128]

Checksum: 0xa4d4 [unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps

[Timestamps]

[SEQ/ACK analysis]

TCP payload (438 bytes)

Hypertext Transfer Protocol

HTTP/1.1 200 OK\r\n

Date: Mon, 06 Oct 2025 16:34:28 GMT\r\n

Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.33 mod_perl/2.0.11 Perl/v5.16.3\r\n

Last-Modified: Mon, 06 Oct 2025 05:59:01 GMT\r\n

ETag: "51-6407725eb5514"\r\n

Accept-Ranges: bytes\r\n

Content-Length: 81\r\n

Keep-Alive: timeout=5, max=100\r\n

Connection: Keep-Alive\r\n

Content-Type: text/html; charset=UTF-8\r\n

\r\n

[Request in frame: 1054]

[Time since request: 0.348895000 seconds]

[Request URI: /wireshark-labs/INTRO-wireshark-file1.html]

[Full request URI: http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html]

File Data: 81 bytes

Line-based text data: text/html (3 lines)

شکل ۷: HTTP response

مراجع

- [1] “Not-having-congestion-in-circuit-switching,” <https://g.co/gemini/share/f34a775fa995>, 2025.
- [2] “Tdm-over-fdm,” <https://g.co/gemini/share/ee20182fcc54>, 2025.
- [3] “San-francisco-to-new-york,” <https://www.distance.to/San-Francisco/New-York>, 2025.